



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE INGENIERÍA

INGENIERIA EN SISTEMAS INTELIGENTES

Interfaces Gráficas con Aplicaciones

Reporte: JUEGO UNO

Alumnos:

Iker Alejandro Arriaga López

María Fernanda Hernández Galván

Regina Escobar Zárate

San Luis Potosí, 11 de octubre del 2026

DESCRIPCIÓN GENERAL DEL PROYECTO

Este proyecto consiste en la creación de una adaptación interactiva del juego de mesa "UNO". El desarrollo de este entorno tiene una interfaz gráfica intuitiva y familiar para el usuario, está diseñado para 3 jugadores en una modalidad de multijugador local, también cuenta con una base de datos sencilla que nos permite la gestión y persistencia de datos como la información de los jugadores, el historial de partidas perdidas o ganadas por cada jugador y todos los movimientos hechos en cada turno.

Todo el sistema está desarrollado en Visual Studio con el marco GUI Windows Forms, con una base de datos modelada en MySQL Workbench.

ARQUITECTURA DEL PROYECTO

▪ Patrón Arquitectónico

- **Tipo de arquitectura:** Arquitectura de Tres Capas/Cliente-Servidor desacoplada
- **Descripción general:** El sistema separa la lógica de la interfaz de usuario, la lógica de negocio y el almacenamiento de datos en componentes independientes que se comunican entre sí mediante servicios web (API) y conectores directos a bases de datos relacionales.

▪ Descripción de las Capas

- **Capa de Presentación y Lógica del Cliente:** Se usó C# (.NET Framework /.NET Core), Windows Forms. Con está tecnología se gestiona la interfaz gráfica del usuario (el tablero, las cartas, los turnos, las "animaciones" y las ventanas de diálogo como el Form3). Contiene las reglas del juego de mesa de UNO (validación de jugadas, efectos de cartas especiales y control de tres jugadores).
- **Capa de Servicios/ API (Backend Intermedio-Python):** Python y FastAPI/Uvicorn se utilizan actuando como un puente de servicios web (API REST). Expone endpoints HTTP (por ejemplo, /jugadores) para procesar peticiones y entregar información estructurada en formato **JSON** al cliente de escritorio cuando se le solicita el ranking de ganadores.

- **Capa de Datos (Almacenamiento-MySQL):** Las tecnologías usadas fueron: MySQL Server, conector MySqlConnector y mysql.connector.
  Almacena de manera persistente la información de los usuarios (jugador), las sesiones activas y finalizadas (partida), las estadísticas acumuladas de victorias y derrotas (historial_partidas) y el registro detallado de las acciones de los usuarios (log_juego).

▪ Flujo de Datos y Comunicación

- **Interacción local:** El usuario realiza una acción en el juego (Form1), el cual interactúa directamente con la base de datos MySQL mediante consultas SQL (INSERT/UPDATE) usando MySqlConnector para guardar logs y registrar estados de la partida en tiempo real.
- **Consulta externa vía API:** Al finalizar la partida, el juego abre la ventana de resumen (Form3), la cual realiza una petición HTTP asíncrona (GET) hacia el servidor local de la API en Python.
- **Respuesta:** La API consulta los datos en MySQL, los serializa a formato JSON y los devuelve al cliente de C# para mostrarlos formateados en pantalla.

▪ Stack Tecnológico

- **Lenguajes de programación, de consulta y manipulación de datos:** C#, Python, SQL.
- **Frameworks y Librerías:** Windows Forms (UI), FastAPI/Uvicorn (API), MySqlConnector & Newtonsoft.Json (Conectividad y serialización).
- **Base de datos:** MySQL
- **Control de versiones:** Git y GitHub (bajo un flujo de trabajo basado en ramas y Pull Request).

DIVISIÓN DE TAREAS

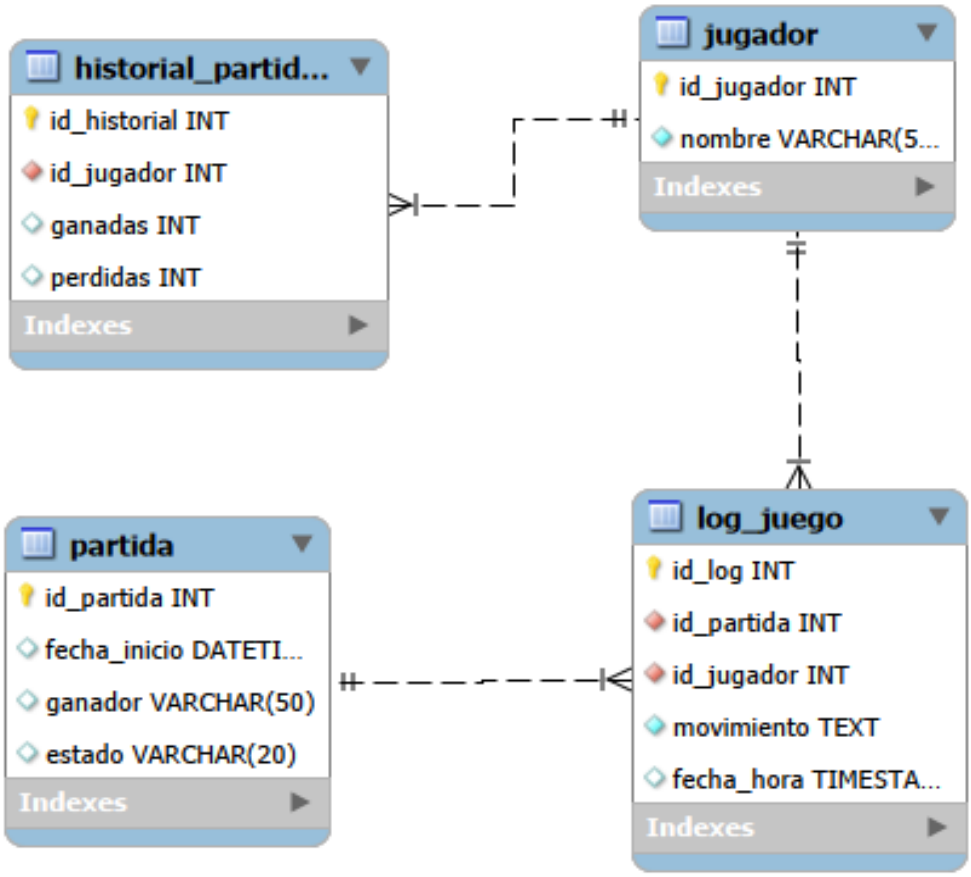
Integrante encargado	Descripción de la tarea
Regina Escobar Zarate / regin4-git	- **Creación de la base de datos y diagrama entidad-relación.** - **Creación de la clase Jugador con atributos y métodos:** La clase jugador se creará con el propósito de tener a los jugadores en la lógica del juego como objetos pudiendo usar sus métodos RecibirCarta(), JugarCarta() y ContarCartas() haciendo más fácil el desarrollo de las jugadas. - **Lógica de los turnos:** Creación de las funciones ActualizaTurnoLabel() y AvanzarTurno(), dichas funciones permitirán llevar un control de los turnos de los 3 jugadores en un ciclo de reinicio cuando se llegue al turno inválido como 4 o -1 dependiendo de la dirección del juego. - **Lógica de las jugadas (Tirada de cartas):** Realizó un evento (CartaClick()) para el momento de hacer click en cualquier carta (se suscriben las cartas al evento cuando se reparten al jugador, cuando este roba del mazo o cuando come un +4 o un +2). Dicho evento validará turno y que sea una jugada válida (mismo color, mismo número, etc). Creación de la función RealizaJugada() que removerá la carta del jugador tanto del tablero como de la lógica y AplicarEfectoCarta() que aplicará los diferentes efectos de las cartas +4, +2, bloqueo, reversa, además de la función DarCartasASiguienteJugador() para cuando alguien tire un +4 o un +2. - **Implementación de ventanas para caso COMODÍN**: Diseño de una pequeña ventana que permite elegir un color de los 4 disponibles para cuando se haya tirado un comodín de color o un +4. - **Implementación del robo desde mazo**: Creación de la función RobarMazoCarta() que permite tomar una carta cuando no tienes cartas válidas. Creación de la función JugadaEstrategica() para cuando se robó una carta del mazo y es válida se muestra una ventana para preguntar al jugador si desea jugarla o desea conservarla.

	- **Reinicio Juego:** Reinicia el juego limpiando los controles de la interfaz y la pila de cartas, también las variables las reinicia (direccionJuego, turnoActual, etc), vuelve a barajar el mazo y a repartir cartas entre jugadores.
Iker Alejandro Arriaga López	- **Implementación de la interfaz de juego:** La interfaz conlleva la distribución y diseño de los controles dentro de la ventana y su integración en la lógica del juego; cada jugador debe poseer un numero de cartas y estas deben verse reflejadas en el tablero; al igual que el tablero debe mostrar las cartas jugadas. La interfaz se separará en 4 secciones distintas: 1. Jugador 1, 2 y 3: Se encargará de mostrar las cartas y nombre correspondientes a cada jugador. 2. Tablero: En esta sección se dibujarán las cartas apiladas y se iran actualizando según vaya transcurriendo el juego. También se agregará un botones para reiniciar el juego y para gritar "¡UNO!". Gran parte del código correspondiente a esta implementación se encontrará en la ventana principal (Form1). - **Creación de la clase Carta:** La clase Carta cumple con la función de representar una carta de juego dentro del código para manipular de forma más eficiente tanto los elementos gráficos que la describen, como su manejo en las reglas internas del juego. Para explicarme mejor, cada carta contendrá un enumerador, "TipoCarta", que determina el tipo de carta que es: - NUMERO: La carta contiene un numero entre 0 y 9. - BLOQUEA: Bloquea el turno del siguiente oponente. - REVERSA: Cambia el sentido del juego. - MASDOS/MASCUATRO: Obliga a robar al jugador 2/4 cartas y también le hace perder su turno. - COMODIN: Permite cambiar el color actual del juego. También se agregará otro enumerador "ColorCarta", que determinará el color de la carta, y en caso de no tenerlo lo generaliza a un valor "ESPECIAL".

	- **Creación de la clase Baraja:** Ya explicado el porqué de la clase Carta, esta clase, Baraja, se encargará de definir cómo usar a Carta dentro de las reglas del juego. Según las reglas oficiales de UNO: - Primero, solo existen dos ejemplares de un mismo número para cada color, a excepción del número 0, que solo cuenta con uno en cada color. - Segundo, las cartas especiales de colores (bloquear, comer dos, cambiar sentido) también cuentan con dos ejemplares para cada color. - Y tercero, solo hay 4 ejemplares para las cartas de comodín y comer cuatro. La clase Baraja se encargará entonces de crear este número concreto de cartas, además de barajear este mazo, y asegurarse de quitar cartas al comer.
Maria Fernanda Hernandez Galvan / f3rhg/ Fernanda	- **Conexión inicial a la base de datos:** Para la conexión inicial a la base de datos se utilizará el código visto en clase, únicamente se verificará que la conexión se haga con éxito, es decir, se implementará y ejecutará un mensaje en pantalla que indique de manera clara que la conexión se estableció correctamente. - **Implementación de registro de jugadores y conexión a la base de datos:** - Para la implementación del registro, se planea diseñar una interfaz/ventana en la cual los jugadores se puedan registrar únicamente con su nombre de usuario. Para llevar a cabo estas acciones, la lógica correspondiente deberá ser implementada en el código de la conexión inicial a la base de datos, considerando los siguientes escenarios: 1. Cuando un jugador acceda al juego por primera vez, se mostrará un mensaje indicándole que es “Nuevo usuario”. 2. Si el jugador ya ha sido registrado, se deberá verificar su existencia en la base de datos y se mostrará un mensaje como recordatorio de que ya estaba registrado como usuario. - **Implementación de la API** - Para la implementación correcta de la API, se tendrá que seguir una serie de pasos:

	1. Lo primero que se debe hacer es establecer la estructura que tendrá nuestra API. Al igual que en la conexión a la base de datos, se implementará utilizando como base el código visto en clase. En este caso se deberá implementar usando el lenguaje Python. 2. Una vez establecida la estructura, se deberá conectar la API a nuestra base de datos utilizando la librería **mysql.connector,** permitiendo crear endpoints (como /jugadores) que ejecuten consultas SQL para extraer y retornar la información de los usuarios en formato JSON. 3. En el código del juego se creará la clase "ApiService" que realizará la petición GET a un endpoint determinado; se implementará utilizando HttpClient debido a que nos pareció la mejor opción de las que se nos presentaron en clase. - **Implementación de la ventana "Resumen de partida"**: Esta ventana se implementará con el fin de verificar que la API se conecte correctamente a la base de datos para obtener la información en formato JSON. - En esta sección del código (en el juego) es donde se ejecutará la petición GET al servidor para traer el JSON, darle un formato legible e imprimirlo en pantalla. Si la petición se realiza con éxito los datos se mostrarán en un ranking ordenando a los jugadores según sus partidas ganadas, si falla mostrará un mensaje de error. - **Implementación de la lógica para verificar que un jugador haya ganado**: Para que un jugador haya ganado deberá haber tirado su última carta y haber dicho "UNO". - **Implementación la lógica para la penalización**: En caso de que un jugador tire su última carta si haber dicho "UNO", se le penalizará. Esta penalización consistirá en sumarle dos cartas a su "mano"

DIAGRAMA ENTIDAD-RELACIÓN



LISTADO DE COMMITS

OCTUBRE

Commit / Referencias	Fecha y Hora	Autor	Descripción
f0d77c9	Sat Oct 10 23:03:16 2026	Iker Alejandro Arriaga López	Merge pull request #21 from Ale-Pez/API - Automatizacion de la instalacion/preparacion de paquetes necesarios para el programa
8d4cc18	Sat Oct 10 23:01:41 2026	Iker Alejandro Arriaga López	Actualizacion de los cambios actuales de Main
2991984	Sat Oct 10	Iker Alejandro	Intento de subida de cambios

	22:54:39 2026	Arriaga López	
2ab7aff	Sat Oct 10 22:32:49 2026	f3rhg	Eliminar __pycache__ del repositorio
86f28c8	Sat Oct 10 22:18:44 2026	f3rhg	Se agrega script iniciar_api.bat para automatizar la API
9aea863	Sat Oct 10 21:55:27 2026	Iker Alejandro Arriaga López	Merge pull request #18 from Ale-Pez/API - Cambios en forms3.cs
e350bdd	Sat Oct 10 21:54:21 2026	f3rhg	Merge branch 'main' of https://github.com/Ale-Pez/UNO-Project into API
7687fe4	Sat Oct 10 21:51:07 2026	f3rhg	cambios en forms3.cs
2fb1afa	Sat Oct 10 18:34:47 2026	Regina Escobar Zarate	Merge pull request #17 from Ale-Pez/mejoras_interfaz - Mejoras interfaz
bea426b	Sat Oct 10 18:26:18 2026	Iker Alejandro Arriaga López	Soluciona error al cambiar de color con una carta numerica. Quita la figura de referencia para la figura de colores
c328ad6	Sat Oct 10 18:08:14 2026	Iker Alejandro Arriaga López	Agrega una figura para indicar el color de carta del juego actual
153416f	Sat Oct 10 16:40:17 2026	Regina Escobar Zarate	Merge pull request #16 from Ale-Pez/mejoras_interfaz - Mejoras interfaz
c30a7a6	Sat Oct 10 12:43:29 2026	Iker Alejandro Arriaga López	Recoloca la etiqueta que emerge al jugar ciertas cartas (ej. +4)
f35e8b2	Thu Oct 8 11:49:40 2026	Iker Alejandro Arriaga López	Los controles del centro (Mazo, Pila, etc.) Han sido centrados. Tambien se han hecho ligeras correcciones de ortografía o código muerto

442d602	Wed Oct 7 19:13:48 2026	Maria Fernanda Hernandez Galvan	Merge pull request #15 from Ale-Pez/feature/CambiosMensajesSpam - Feature/cambios mensajes spam
ff0949e	Wed Oct 7 19:02:12 2026	regina4-git	Ignorar carpeta __pycache__ en .gitignore
810fac3	Wed Oct 7 18:50:28 2026	regina4-git	Modificacion mensajes Spam a Label con timer
e6a35a5	Wed Oct 7 17:00:27 2026	Regina Escobar Zarate	Merge pull request #14 from Ale-Pez/API - Modificacion de la interfaz "Registro"
7958065	Tue Oct 6 11:53:11 2026	f3rhg	Modificacion de la interfaz "Registro"
d2e6486	Mon Oct 5 06:29:04 2026	Iker Alejandro Arriaga López	Merge pull request #13 from Ale-Pez/feature/ReinicioJuego-BarajeoPilaCartas - Reinicio del juego y barajar la pila de cartas
ccc93e5	Sun Oct 4 20:31:12 2026	regina4-git	Reinicio de juego en Form3
0eddbde	Sun Oct 4 20:00:34 2026	regina4-git	Reinicio de juego en Form1
ab84444	Sun Oct 4 19:08:25 2026	regina4-git	Barajar pila de cartas
782743b	Sun Oct 4 18:18:47 2026	Regina Escobar Zarate	Merge pull request #12 from Ale-Pez/API - Se adjuntó el archivo que contiene la API al proyecto
00176b3	Sun Oct 4 14:37:14 2026	f3rhg	Se adjuntó el archivo que contiene la API al proyecto
74a7330	Sun Oct 4 14:31:45 2026	Iker Alejandro Arriaga López	Merge pull request #11 from Ale-Pez/API - Ajustes en la ventana de "resumen de partida" y conexión de la API a C#
3f0d984	Sun Oct 4 14:27:55 2026	f3rhg	Ajustes en la ventana de "resumen de partida" y conexion de la API a c#

21c2c39	Sun Oct 4 12:44:41 2026	Maria Fernanda Hernandez Galvan	Merge pull request #10 from Ale-Pez/feature/cambio-002-bd - Segunda actualizacion de la BD, el juego ya guarda los datos en la BD
b05e63a	Sun Oct 4 12:38:57 2026	regina4-git	Segunda actualizacion de la BD, el juego ya guarda los datos en la BD
0593804	Sun Oct 4 10:59:32 2026	Iker Alejandro Arriaga López	Implementación de la conexión a la API para conectar con la base de datos
205d75f	Sun Oct 4 03:15:59 2026	f3rhg	correciones en la logica de penalizacion
7799c7b	Sun Oct 4 02:44:28 2026	f3rhg	Implementacion de la penalizacion (si el jugador no dice UNO antes de tirar su ultima carta se agregan dos cartas a su panel)
292ca7a	Sat Oct 3 23:31:02 2026	f3rhg	implementacion del metodo "verificar ganador"
6d69b09	Sat Oct 3 22:52:46 2026	f3rhg	Implementacion de la interfaz "resumen de partida" que obtiene el formato json de la API
b1102da	Sat Oct 3 21:41:49 2026	f3rhg	Creacion de la clase apiService (puente de comunicación entre C# y la api)
8870af2	Sat Oct 3 20:39:14 2026	Regina Escobar Zarate	Merge pull request #8 from Ale-Pez/feature/boton_uno - Correción de detalles de la interfaz
e9637b4	Sat Oct 3 20:25:31 2026	Iker Alejandro Arriaga López	Comprobación de que la primer carta de la pila sea únicamente numérica
6c30212	Sat Oct 3 20:19:26 2026	Iker Alejandro Arriaga López	Agrega el boton de UNO y lo oculta al iniciar la ventana Form1
6986b43	Sat Oct 3 19:45:08 2026	f3rhg	Merge pull request #7 from Ale-Pez/feature/TiradaDeCartas-AgregarALogJuego - Feature/tirada de cartas agregar a log juego
6dbac51	Sat Oct 3 19:22:13 2026	regina4-git	Actualizar log-juego y registrar-jugador

49804ea	Sat Oct 3 18:41:11 2026	regina4-git	Funcion para escoger color del comodin y robar del mazo restante
850612d	Sat Oct 3 16:02:14 2026	regina4-git	Evento tira carta, Funcion valida turno y Aplicar efecto de carta
5b3f793	Sat Oct 3 14:16:01 2026	regina4-git	Verificar Jugada Valida y Logica de turnos: Avanzar y Actualizar
f89a99c	Sat Oct 3 11:59:40 2026	Iker Alejandro Arriaga López	Merge pull request #6 from Ale-Pez/feature/cambio-001-bd - Base de datos corregida en historial_partida y log_juego
6c2cacd	Sat Oct 3 11:44:53 2026	regina4-git	Base de datos corregida en historial_partida y log_juego
30d638f	Thu Oct 1 11:10:24 2026	Iker Alejandro Arriaga López	Merge pull request #5 from Ale-Pez/ConexionBD - Conexion BD
b8553f7	Thu Oct 1 11:07:24 2026	Iker Alejandro Arriaga López	Merge pull request #4 from Ale-Pez/feature/clase-Jugador - Implementacion de la clase Jugador y actualizacion de Repartir_Cartas

SEPTIEMBRE

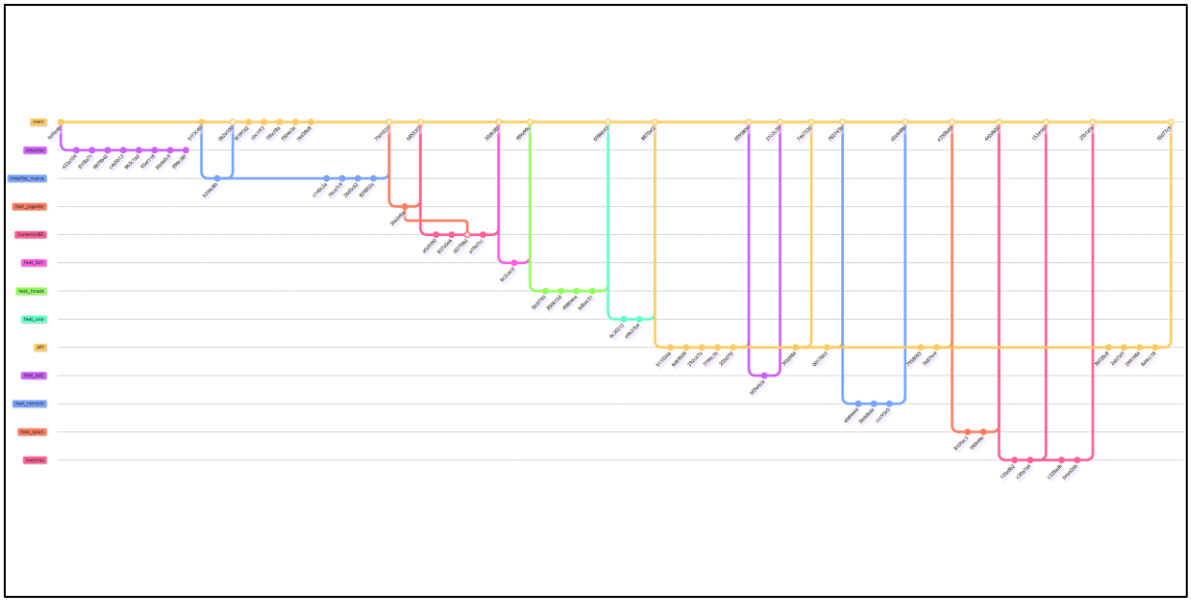
Commit / Referencias	**Fecha**	**Autor**	**Descripción**
a19e7cc	Wed Sep 30 17:29:31 2026	Fernanda	Implementacion de registro de jugadores y base de datos
d2f706d	Wed Sep 30 12:53:18 2026	Fernanda	Merge remote-tracking branch 'origin/feature/clase-Jugador' into ConexionBD
3ba2eba *(origin/feature/clase-Jugador)*	Mon Sep 28 19:50:33 2026	Regina	Implementacion de la clase Jugador y actualizacion de Repartir_Cartas
75d1031	Sun Sep 27 17:44:49 2026	Iker Alejandro Arriaga López	Merge pull request #3 from Ale-Pez/interfaz_nueva - Mejoras en la interfaz

829852a *(origin/interfaz_nueva, interfaz_nueva)*	Sun Sep 27 17:41:38 2026	Iker Alejandro Arriaga López	Enseña las cartas correspondientes a cada jugador y la pila
26d5c62	Sun Sep 27 16:54:30 2026	Iker Alejandro Arriaga López	Implementa logica bajo la pila de cartas
16ca7c9	Sun Sep 27 16:41:18 2026	Iker Alejandro Arriaga López	Reparte las cartas inciales a los tres jugadores
c140c2a	Sun Sep 27 16:35:18 2026	Iker Alejandro Arriaga López	Modifica la interfaz para ajustarse mejor a los elementos utilizados
837a5e6	Sat Sep 26 19:57:19 2026	f3rhg	Update Program.cs
afa9390	Sat Sep 26 19:48:54 2026	Fernanda	Conexion a la base de datos
16d38e8	Sat Sep 26 17:09:48 2026	Iker Alejandro Arriaga López	Cambio de ubicacion de las clases Baraja y Carta
f504e3e	Sat Sep 26 17:08:31 2026	Iker Alejandro Arriaga López	Agrega la baraja que se usara en el programa con su inicializacion y reparticion de cartas (solo al jugador 1)
f9bcf8a	Sat Sep 26 16:39:43 2026	Iker Alejandro Arriaga López	Implementa metodo para robar cartas de la baraja
cbc1ff3	Sat Sep 26 16:35:49 2026	Iker Alejandro Arriaga López	Agrega metodo para Barajar cartas
8f397d2	Sat Sep 26 16:25:54 2026	Iker Alejandro Arriaga López	Agrega la inicializacion de las cartas

062a100	Sat Sep 26 15:57:21 2026	Iker Alejandro Arriaga López	Merge pull request #2 from Ale-Pez/interfaz_nueva - Agrega interfaz y clases del juego
b336c8b	Sat Sep 26 15:53:25 2026	Iker Alejandro Arriaga López	Agrega interfaz y clases del juego
096c38f *(interfaz)*	Sat Sep 26 15:24:58 2026	Iker Alejandro Arriaga López	Agrega .gitignore y elimina archivos innecesarios de Visual Studio
bbda0c9	Sat Sep 26 15:15:58 2026	Iker Alejandro Arriaga López	Agrega archivos de clase Carta y Baraja
93ef118	Sat Sep 26 15:14:46 2026	Iker Alejandro Arriaga López	Agrega clase Baraja que guardara todas las cartas disponibles del juego
863c1ad *(origin/interfaz)*	Fri Sep 25 16:00:41 2026	Iker Alejandro Arriaga López	Manejo de errores en parametros y numeros fuera del rango
c4a5b13	Fri Sep 25 15:44:10 2026	Iker Alejandro Arriaga López	Agrega getters y setters de Carta
9879b42	Fri Sep 25 15:41:13 2026	Iker Alejandro Arriaga López	Agrega constructores a Carta
81f8a71	Fri Sep 25 15:37:53 2026	Iker Alejandro Arriaga López	Base de la clase Carta. Generacion de enumeradores y atributos
433a104	Fri Sep 25 15:25:06 2026	Iker Alejandro Arriaga López	Plantilla de la interfaz
b1f3c40	Wed Sep 23 16:07:44 2026	Regina	Agrega script de la base de datos
feffe4b	Tue Sep 22	Iker Alejandro	Plantilla del proyecto

	11:18:51 2026	Arriaga López	

LISTADO DE RAMAS GENERADAS



PROBLEMAS ENCONTRADOS Y SOLUCIONES

Descripción del problema	Solución
Iker: Al querer hacer merge desde una rama llamada “interfaz” a main GitHub no podía realizar la combinación debido a un desfaz de versiones del main en linea y remoto.	Cree otra rama generada desde el main actualizado y manualmente copie los cambios hechos en mi rama en la nueva.
Iker: El único detalle con la base de datos fue que, como fue creada por mi compañera Regina, tuve que obtener el script y configurarla de forma local; aunque este proceso fue un poco tedioso, era totalmente necesario para comprobar que la conexión y el registro se realizaran con éxito.	Para resolverlo, tomé el script, y lo ejecuté paso a paso en MySQL para estructurar correctamente las tablas correspondientes, esto se tuvo que hacer varias veces porque no funcionaba.
Fernanda: Tanto Regina como yo no nos veíamos reflejadas como contribuidores del	Esto fue un problema que como tal no pudimos solucionar, debido a que creamos

proyecto en GitHub, a pesar de que se vean reflejados nuestros cambios y commits.	usuarios “ficticios” en una de las prácticas en clase. Nuestros usuarios no existen como cuentas reales en GitHub, por lo que, si aparecen los cambios, pero GitHub no nos reconoce como usuarios de su plataforma. La solución fue volver a declarar nuestro nombre y correo en Git, pero eso solo soluciono que los cambios futuros salieran a nuestra cuenta real. Por suerte, nos dimos cuenta antes de realizar cambios fuertes, por lo que las métricas de GitHub son bastante acertadas al trabajo final.
Regina: Problemas al intentar hacer git pull origin main o git checkout main, el problema se debía a archivos generados localmente dentro de la carpeta __pycache__ mientras se ejecutaba la API.	Para resolver el problema se agregó los archivos que se querían ignorar al archivo .gitignore